Trivia Wizards — Full Presentation Scripts & Slide Guide

> Total time: ~10 minutes | 5 speakers | Yaseer demos at the end Each slide note describes what should be on the slide (text, visuals, code). Speaker script is what you say aloud.

DANIEL — Intro & Project Overview (1.5 min)

Slide 1 — Title Slide

Slide content:

- Large title: **Trivia Wizards**
- Subtitle: *A Real-Time Multiplayer Bar Trivia Platform*
- Team names: Daniel · Ben · Timothy · Mariyem · Yaseer
- Semester/course info
- (Optional) neon/dark background matching the app aesthetic

Script:

> "Hi everyone. We're Team Trivia Wizards, and what we built is a real-time, multiplayer trivia game platform designed for bars and venues. Think of it as a fully digital version of bar trivia night — no paper scoresheets, no manual scoring, and no waiting around. Everything happens live."

Slide 2 — The Problem

Slide content:

- Heading: **The Problem**
- 3 bullet points:
  - Traditional bar trivia relies on paper answer sheets and manual score tallying
  - Score disputes, delays between rounds, and no real-time feedback for players
  - Hosts have no tool to manage questions, rounds, and scoring from one place

Script:

> "Bar trivia nights are popular, but they're still mostly run with pen and paper. Hosts spend time collecting sheets, tallying scores manually, and players just sit and wait. There's no instant feedback, no live leaderboard, and no clean way for a host to manage everything in one place. We built Trivia Wizards to solve exactly that."

Slide 3 — Our Solution: Three Interfaces

Slide content:

- Heading: **Our Solution**
- Three columns or icons, each labeled:
  - **Host Dashboard** — `/dashboard` — Create sessions, manage rounds, control the game
  - **Player Kiosk** — `/kiosk` — Teams register and submit answers from any device
  - **TV Display** — `/display/[sessionId]` — Read-only leaderboard and question screen for the room
- Small diagram showing how the three connect (arrows from Host → Kiosk and Host → TV Display)

Script:

> "Our solution has three interfaces that work together. The Host Control Panel is where the game organizer creates sessions, builds rounds, and controls the flow of the game. The Player Kiosk is a touch-friendly screen at each table — teams register there and submit their answers. And the TV Display is what the whole room sees — it shows the current question, the countdown timer, and the live leaderboard. All three update in real time, simultaneously."

Slide 4 — The Game Flow

Slide content:

- Heading: **How a Game Works**
- Numbered flow diagram (horizontal or vertical):
  1. Host creates a session and adds rounds with questions
  2. Teams scan a QR code to register at their table's kiosk
  3. Host starts the game → rounds begin
  4. Each question: countdown timer runs, teams submit answers
  5. Host reveals correct answer → scores update live
  6. After all rounds → podium screen shows 1st, 2nd, 3rd place

Script:

> "Here's the full game flow. The host creates a session and loads it up with rounds and questions from the question bank. Teams join by scanning a QR code — they land on the kiosk, type their team name, and they're in. The host starts the game, and from there it's question by question: a countdown timer runs, teams submit their answer before time's up, and when the host hits reveal, everyone sees the correct answer and the scores update instantly across every device in the room. When the last round is done, a podium screen automatically shows the top three teams. The whole thing runs without any manual score tracking."

BEN — Tech Stack & Architecture (2 min)

Slide 5 — Tech Stack Overview

Slide content:

- Heading: **Tech Stack**
- Two-column layout or icon grid:
  - **Frontend**: Next.js 16 (App Router) · React 19 · TypeScript 5 · Tailwind CSS 4
  - **Backend**: Next.js API Routes · Custom Node.js HTTP server
  - **Real-Time**: Socket.IO 4.8 (WebSocket-only)
  - **Database**: Prisma 7 ORM · Supabase PostgreSQL
  - **Auth**: iron-session (hosts) · JWT via JOSE (kiosks)
  - **Validation**: Zod 4

Script:

> "Let me walk through the stack. On the frontend we're using Next.js 16 with the App Router, React 19, TypeScript throughout, and Tailwind CSS 4 for styling. On the backend, Next.js API Routes handle our REST endpoints, but we run a custom Node.js HTTP server on top of Next.js — that's what lets us attach Socket.IO for real-time communication. Our database is Supabase PostgreSQL, accessed through Prisma 7 as our ORM, with connection pooling for production. For auth, hosts log in with a password and get a secure cookie via iron-session. Kiosk devices get a 30-day JWT token so they stay identified across sessions without needing to log in again."

Slide 6 — Real-Time Architecture

Slide content:

- Heading: **Real-Time Architecture: Socket.IO**
- Diagram showing three client types (Host, Kiosk, TV Display) connected via arrows to a central Socket.IO Server
- Three room labels:
  - `session:{sessionId}` — all clients
  - `host:{sessionId}` — host only
  - `kiosk:{teamId}` — per-team
- Below diagram: one callout box — **"WebSocket-only. HTTP polling disabled."**

Script:

> "Real-time communication is the core of this app — everything has to stay in sync. We use Socket.IO with three types of rooms. The session room broadcasts to everyone in that game: the host, all kiosks, and the TV display. The host room is private — things like score overrides only go there. And each team gets their own kiosk room for answer acknowledgements. One important architectural choice: we disabled HTTP long-polling entirely. Socket.IO defaults to polling first and upgrading to WebSocket, but polling adds one to three seconds of latency on mobile. By forcing WebSocket-only from the start, timer ticks and question delivery are instant on any device."

Slide 7 — Key Architectural Decisions

Slide content:

- Heading: **Key Design Decisions**
- Three callout cards:
  1. **Server-Side Timer** — Countdown runs on the server, broadcast every second via `timer:tick` — prevents client clock drift and cheating
  2. **WebSocket-Only Transport** — `transports: ["websocket"]` in `socket-client.ts` — eliminates 1–3s polling latency on mobile
  3. **Synthesized Audio** — Web Audio API oscillators, no audio files — zero asset management, instant sound on any event

Script:

> "Three decisions stand out architecturally. First, the game timer runs entirely server-side. The server emits a timer:tick event every second to all clients. This means every screen — host panel, kiosk, TV display — sees the exact same countdown, and players can't manipulate time on their device. Second, as I mentioned, WebSocket-only. Third — and this one's fun — all sound effects are synthesized in real time using the Web Audio API. There are no audio files anywhere in the project. The countdown ticks, the fanfare when a round starts, the wrong-answer wah-wah — all of it is generated mathematically from oscillators and gain nodes. That means zero asset management and instant audio on any event."

TIMOTHY — Database & Game Engine (2 min)

Slide 8 — Database Schema

Slide content:

- Heading: **Database Schema (Prisma + Supabase)**
- Entity diagram or clean nested list:

```
Venue
├── Tables (physical table seats at the venue)
├── Questions (question bank — options stored as JSON)
└── GameSessions
    ├── Teams (registered per session, holds totalScore)
    └── Rounds
        └── RoundQuestions (ordered, has timeLimit)
            └── AnswerSubmissions (isCorrect, pointsAwarded, submittedAt)
```

- Below: Two enum badges — `SessionStatus: LOBBY | ACTIVE | COMPLETED` and `RoundStatus: PENDING | ACTIVE | COMPLETED`

Script:

> "Let's talk about the data layer. At the top level we have a Venue, which owns Tables — those are the physical table spots in the bar — and a Question bank. A Venue also has GameSessions. Each session holds Teams and Rounds. Rounds contain RoundQuestions, which link a question to a round with a

> specific time limit and order index. And each RoundQuestion has AnswerSubmissions — one per team — which record whether the answer was correct, how many points were awarded, and when it was submitted. Questions store their multiple-choice options as a JSON array in the database, which keeps the schema flexible."

Slide 9 — State Machine

Slide content:

- Heading: **Session & Round State Machine**
- Two state flow diagrams side by side:

  Session: `LOBBY` → `ACTIVE` → `COMPLETED`

  Round: `PENDING` → `ACTIVE` → `COMPLETED`
- Below: Small note — "State transitions triggered by host via Socket.IO events and persisted to Prisma"

Script:

> "We model the game as a state machine at two levels. A Session moves from LOBBY — where teams join — to ACTIVE when the game starts, and finally to COMPLETED when the host ends it. Rounds follow the same pattern: PENDING before the host starts them, ACTIVE while questions are running, and COMPLETED once the host wraps up. Every state transition is triggered by a host socket event — like `host:start-round` or `host:end-round` — and immediately persisted to the database through Prisma. That means if you refresh any screen mid-game, it comes back to the correct state."

Slide 10 — Game Engine & Socket Events

Slide content:

- Heading: **Game Engine & Event Flow**
- Left side — code snippet or pseudocode block:

```
// lib/game-engine.ts
startTimer(roundQuestionId, seconds, io)
  → emits timer:tick every 1s
  → emits timer:expired at 0

overrideTimer(roundQuestionId, newSeconds)
stopAllSessionTimers(sessionId)
```

- Right side — event flow list (arrows):

```
host:start-question
  → game:question-start  (all clients)
  → timer:tick × N       (every second)
```

```
    → timer:expired
  host:reveal-answer
    → game:answer-reveal   (correct answer + scores)
```

Script:

> "The game engine lives in `lib/game-engine.ts` and it's a singleton that manages all active timers. When the host fires `host:start-question`, the server grabs that question from the database, broadcasts `game:question-start` to every client in the session, and starts a server-side countdown. Every second, it emits `timer:tick` with the remaining seconds. At zero, it emits `timer:expired` and locks out any new answer submissions. The host then manually triggers reveal, which calls `host:reveal-answer` — the server stops the timer, looks up all submissions, calculates scores, and broadcasts `game:answer-reveal` with the correct answer and the updated leaderboard. All of this is fully typed — we have a dedicated `types/socket-events.ts` file that defines every event payload for both directions."

MARIYEM — UI/UX & Design System (1.5 min)

Slide 11 — Design System: Neon Grimoire

Slide content:

- Heading: **Neon Grimoire Design System**
- Color swatches with labels (pull from the app's actual colors):
  - Deep dark background
  - Neon purple / violet (primary)
  - Cyan / teal (secondary)
  - Green (correct answers)
  - Pink / magenta (accent)
- Two or three UI screenshots side by side (host dashboard, kiosk question screen, TV display leaderboard)
- Caption: "Consistent neon glow aesthetic across all three interfaces"

Script:

> "We built a custom design system called Neon Grimoire that runs across all three interfaces through shared CSS custom properties. The palette is dark backgrounds with neon accents — purple and violet as the primary colors, cyan for secondary elements, green for correct answers, and pink for highlights. Every interface — whether you're the host on a laptop, a player on a tablet kiosk, or watching the TV display — feels like it belongs to the same game. We use entrance animations, background gradients, and glow effects consistently so the whole experience feels polished and cohesive."

Slide 12 — Host Dashboard & Session Control

Slide content:

- Heading: **Host Interface**
- Screenshot or mockup of the Host Dashboard (session list with status badges)
- Screenshot or mockup of the Session Control page showing:
  - Rounds list
  - Active question with timer bar
  - A/B/C/D colored buttons
  - QR code
- Two callout labels: "Category filter for question bank" and "QR code → kiosk join link"

Script:

> "The host interface has two main views. The dashboard shows all sessions with their current status — LOBBY, ACTIVE, or COMPLETED — along with the team count and round count at a glance. When the host opens a session, they get the full control panel: they can build rounds, pick questions from the question bank filtered by category, and see the live leaderboard updating as teams answer. A QR code is generated automatically — hosts just point the room at it and teams scan to join. During a question, the host sees the same countdown the players see, and has buttons to reveal the answer and move to the next question."

Slide 13 — Player Kiosk & TV Display

Slide content:

- Heading: **Player & Audience Interfaces**
- Left side — Kiosk screenshots (register screen + question screen with A/B/C/D buttons)
  - Label: "Color-coded buttons · Fullscreen on mobile · Real-time answer feedback"
- Right side — TV Display screenshots (lobby with QR code + leaderboard + podium)
  - Label: "Read-only · Updates live · Podium animation on game end"

Script:

> "The kiosk is designed for tablets or phones sitting at each table. When players first land on it, it goes fullscreen automatically. They enter their team name, hit register, and they're in the lobby waiting for the game to start. During a question, they see four large color-coded answer buttons — purple, cyan, green, and pink for A, B, C, and D. Once they tap an answer it locks, the button dims, and they get instant audio and visual feedback when the host reveals the answer — green glow for correct, red for wrong. On the other side, the TV display is purely read-only — it shows the current question and options during play, the live leaderboard after each reveal, and a podium screen with first, second, and third place when the game ends — complete with a fanfare sound effect."

YASEER — Live Demo (3 min)

Slide 14 — Demo Setup (shown while Yaseer gets tabs ready)

Slide content:

- Heading: **Live Demo**
- Three columns showing what each tab will show:
  - Tab 1: **Host Dashboard** — `localhost:3000/dashboard`
  - Tab 2: **TV Display** — `localhost:3000/display/[sessionId]`
  - Tab 3: **Player Kiosk** — `localhost:3000/kiosk?session=[sessionId]`
- Bottom note: "All three screens update in real-time via WebSocket"

Script (transition line from Mariyem):

> "Now Yaseer is going to show you all of this working live."

Slide 15 — [STAYS ON SCREEN DURING DEMO] — Architecture Reminder

Slide content:

- Heading: **What You're Watching**
- Three-box diagram (can stay visible behind the live demo or on a secondary monitor):
  - Box 1: Host Dashboard → controls the game
  - Box 2: TV Display → what the room sees
  - Box 3: Player Kiosk → what each table uses
- Arrow: All connected via Socket.IO WebSocket

Yaseer's Demo Script (spoken while demonstrating):

> **[Step 1 — Show Host Dashboard]** "I'm logged into the host dashboard. You can see our sessions here — each one shows its status and how many teams and rounds it has. I'm going to open this session."

> **[Step 2 — Open TV Display in second tab]** "I'll switch to the TV display tab — this is what the big screen in the bar would show. Right now it's showing the lobby and a QR code. Teams would scan this to join."

> **[Step 3 — Open Kiosk in third tab or phone]** "I'll open the kiosk — this simulates a player at their table. I'll type in a team name... and register. We're in. You can see the team appear on the host panel and on the TV display instantly — that's the WebSocket connection live."

> **[Step 4 — Start a round and question from Host]** "Back on the host panel, I'll start a round. Now I'll start a question. Watch all three screens — the same countdown timer is ticking on the host panel, the kiosk, and the TV display simultaneously. That's the server-side timer emitting a tick event every second."

> **[Step 5 — Submit answer from Kiosk]** "I'll submit an answer from the kiosk now — I'll pick B. The button locks immediately. Now I'll hit reveal on the host panel. The correct answer highlights on all three screens, the score updates on the leaderboard, and you can hear the sound effect — that's synthesized in real time using the Web Audio API, no audio files."

> **[Step 6 — Show leaderboard update]** "The leaderboard on the TV display updated live. The host panel shows the same scores. Everything is in sync with no page refreshes."

> **[Step 7 — End session and show podium]** "Finally, I'll end the session. And there's the podium — first, second, and third place, with the game-over fanfare. That's the full game loop from start to finish."

Slide 16 — Wrap-Up

Slide content:

- Heading: **What We Built**
- Quick recap bullets:
  - Real-time multiplayer trivia — WebSocket sync across host, kiosk, and display
  - Server-side game engine with typed Socket.IO events
  - Prisma + Supabase database with full state persistence
  - Neon Grimoire design system across three responsive interfaces
  - Synthesized audio, QR code kiosk join, category-filtered question bank
- Bottom: **"Questions?"**

Script (any team member):

> "To wrap up — Trivia Wizards is a production-ready trivia platform with real-time multiplayer, a server-managed game engine, a fully relational database, and a polished design system. We're happy to take any questions."

APPENDIX — Slide Summary Table

Slide	Title	Speaker
1	Title Slide	Daniel
2	The Problem	Daniel
3	Our Solution: Three Interfaces	Daniel
4	The Game Flow	Daniel
5	Tech Stack Overview	Ben
6	Real-Time Architecture	Ben
7	Key Design Decisions	Ben
8	Database Schema	Timothy
9	State Machine	Timothy
10	Game Engine & Event Flow	Timothy
11	Neon Grimoire Design System	Mariyem
12	Host Dashboard & Session Control	Mariyem
13	Player Kiosk & TV Display	Mariyem

Slide	Title	Speaker
14	Demo Setup	Yaseer
15	What You're Watching (stays up during demo)	Yaseer
16	Wrap-Up	Any

Total slides: 16 Total time: ~10 minutes